

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 1 Part A — Uninformed Search Algorithms

Student Name: _Joshua Elvis Hanson_

Student ID: _27372028_

GitHub Repository Link:

_____https://github.com/JoshuaHanson69/Joshua_Hanson_Lab1.git_____

Date Submitted: _06/20/2026_

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	15%
Did you attach evidence of AI use where applicable?	Yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Claude (Anthropic)	o help me understand the lecture concepts, make sense of what each lab task required, and interpret the error messages I encountered while coding.	can you briefly explain what the whole assignment is about and what guidelines i have to follow the setups and imports what are they I tried making my custom map just going straight down column 0 from (0,0) to (14,0) then right to	Conceptual understanding of search algorithms, understanding of heuristic admissibility and consistency, design approach for custom maps, and scaling of custom map 2 in Part B from a 6x6 to a 20x20 grid.	I ran all provided self-check cells to verify my implementations were correct. I manually traced through algorithm logic on small examples. I tested custom maps by running all four algorithms and comparing results. I wrote all reflection question answers

		(14,14). It found a path but it looks boring. Would it be better to make it zigzag so the algorithms have more decisions to make? <i>for the todo 7 for PartA, how should i go about it"</i> (DFS question) <i>what concepts should i learn to understand this"</i> (about DFS) <i>check this out and see if there are errors"</i> (BFS implementation review) I designed a 6x6 U-shaped map myself and I want you to just scale it up to a 20x20, generate for me		in my own words after understanding the concepts explained.

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	N/A	
Prompt history	Yes	AI Use Declaration Form.docx

Screenshot of AI interaction	Yes	AI_Use_Declaration_Form.docx
Exported AI conversation	N/A	
AI-generated code snippet	Yes	AI_Use_Declaration_Form.docx
Revised version showing student input	N/A	

4. Student Declaration

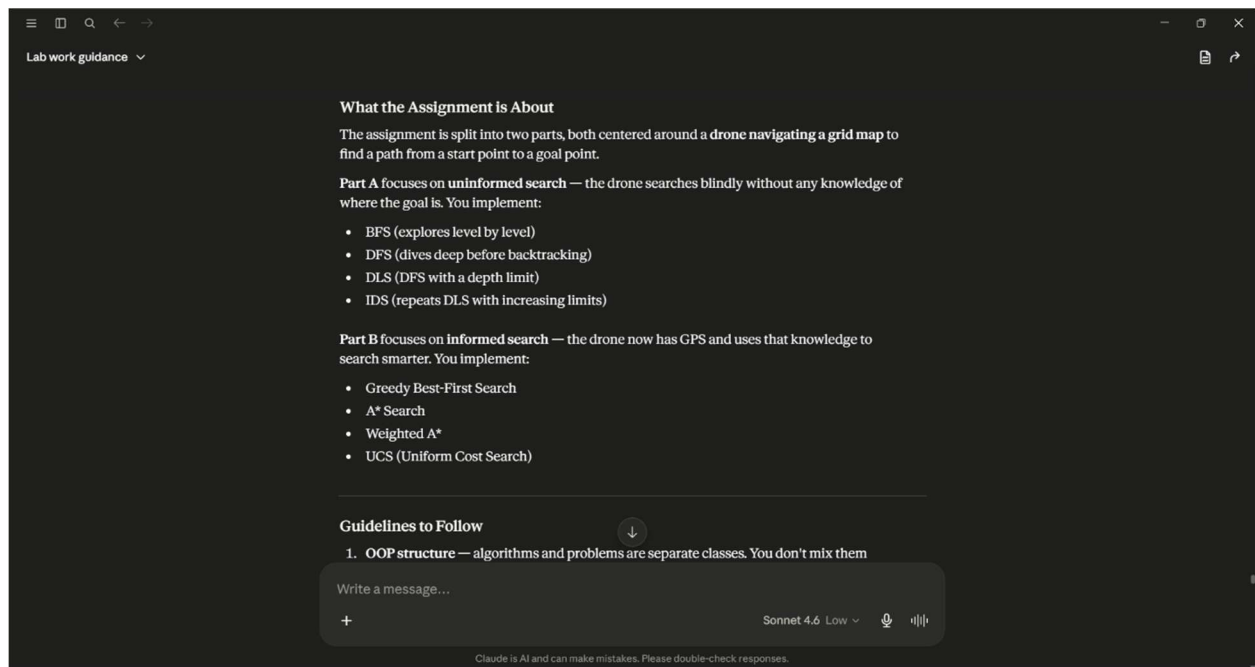
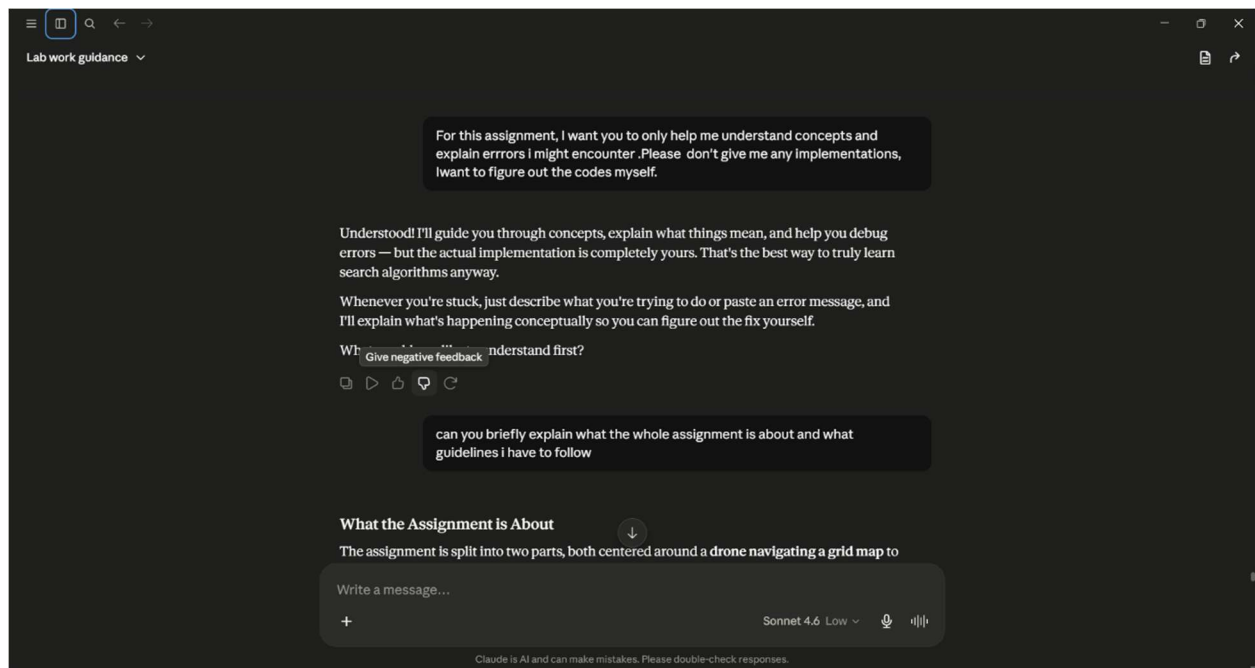
I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	YES
Any use of AI tools has been clearly declared in this form.	YES
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	YES
I reviewed, tested, edited, and improved any AI-generated content before submission.	YES
My AI usage does not exceed 25% of the entire work.	YES
I understand that undeclared or excessive AI use may be treated as academic misconduct.	YES

Student Signature: _____ J.E.H _____

Date: _____ 06/20/2026 _____

SCREENSHOTS



☰

🔍

←

→

Lab work guidance

📄

↻

Guidelines to Follow

1. OOP structure — algorithms and problems are separate classes. You don't mix them
2. Complete all TODOs — each method has a specific job, read the comments carefully
3. Run self-checks — each section has a test cell to verify your implementation
4. Create custom maps — at least two, with results, visualisations, and a short interpretation
5. Answer reflection questions — these test your conceptual understanding
6. Submit via GitHub — public repository, correct filename format

Does that give you a clear picture? What do you want to dig into first?

📄 ▶ 📄 📄 📄 📄

the setups and imports what are they

The Setup and Imports

The imports at the top of the notebook bring in tools you need throughout the lab. Here's what each one does:

Standard Python tools:

- `from abc import ABC, abstractmethod` — ↓ you create **abstract classes**, meaning classes that define a blueprint but can't be used directly. Think of `Problem` as a template

Write a message...

+Sonnet 4.6 Low🔊🔊🔊

Claude is AI and can make mistakes. Please double-check responses.

☰

🔍

←

→

Lab work guidance

📄

↻

📄 ▶ 📄 📄 📄 📄

the setups and imports what are they

The Setup and Imports

The imports at the top of the notebook bring in tools you need throughout the lab. Here's what each one does:

Standard Python tools:

- `from abc import ABC, abstractmethod` — lets you create **abstract classes**, meaning classes that define a blueprint but can't be used directly. Think of `Problem` as a template that `GridProblem` fills in
- `from collections import deque` — gives you a **double-ended queue**, which is what BFS needs. A regular list is slow for this purpose
- `from typing import List, Tuple, Any` — these are just **type hints**, telling you what kind of data a function expects. `List[int]` means a list of integers, `Tuple[int, int]` means a coordinate like `(row, col)`
- `import heapq` — gives you a **priority queue**, used in informed search to always pick the node with the lowest cost first

Data and visualisation:

- `import numpy as np` — for working with **grid arrays** efficiently

Write a message...

+Sonnet 4.6 Low🔊🔊🔊

Claude is AI and can make mistakes. Please double-check responses.

The screenshot shows the Claude AI web interface. At the top, there's a navigation bar with icons for menu, search, and navigation. Below it, a header bar says "Lab work guidance" with a dropdown arrow. The main chat area has a dark background. The user's message is: "IDS search requirements: 1 — Track maximum frontier size. This measures the space complexity of the search. Since IDS uses depth-first traversal, its memory footprint is linear — $O(bd)$ where b is branching factor and d is depth. Track the peak stack depth observed across all iterations. 2 — Success termination condition. When DLS returns `status="success"`, IDS has found an optimal solution at the shallowest depth bound. This triggers early termination — the search is complete and cost-optimal on uniform-cost graphs. 3 — Failure termination condition. `status="failure"` indicates the search space is exhausted — no solution exists at any depth. This means the problem is unsolvable, so continuing to increase the depth bound would be computationally wasteful. 4 — Cutoff termination condition. `status="cutoff"` means the goal exists but lies beyond the current depth bound. If every iteration up to `max_depth` returns cutoff, IDS returns cutoff — indicating the solution exists but exceeds the maximum search horizon." The assistant's response is partially visible, starting with "1 — Track maximum frontier size". At the bottom, there's a text input field with the placeholder "Write a message...", a "Send" button, and a status bar showing "Sonnet 4.6 Low" and various icons.

Lab work guidance

use technical ai terms

Breaking Down Each Requirement

Requirement 1 — Iteration_log

This is a data structure that maintains a log of each iterative deepening cycle. Each entry records the depth bound and the termination status of that DLS call. It enables post-hoc analysis of the search trajectory.

Requirement 2 — Accumulate nodes expanded

IDS has cumulative computational complexity across iterations. Since each iteration restarts from the root, nodes near the top of the search tree are re-expanded multiple times. Your counter must aggregate the total node expansions across all DLS iterations, not just the final one.

Requirement 3 — Track maximum frontier size

This measures the space complexity of the search. Since IDS uses depth-first traversal, its memory footprint is linear — $O(bd)$ where b is branching factor and d is depth. Track the peak stack depth observed across all iterations.

Write a message...

Sonnet 4.6 Low

Claude is AI and can make mistakes. Please double-check responses.

Lab work guidance

```
# 4. If a DLS run returns success, return a SearchResult for IDS.
# 5. If a DLS run returns failure, IDS can stop early and return failure.
# 6. If all limits return cutoff up to max_depth, return cutoff.
#
# Hint:
# dls = DepthLimitedSearch()
# result = dls.search(problem, limit=limit)
```

for this TODO, can you tell me what each requirement means in simple terms pls

Show less

Breaking Down Each Requirement

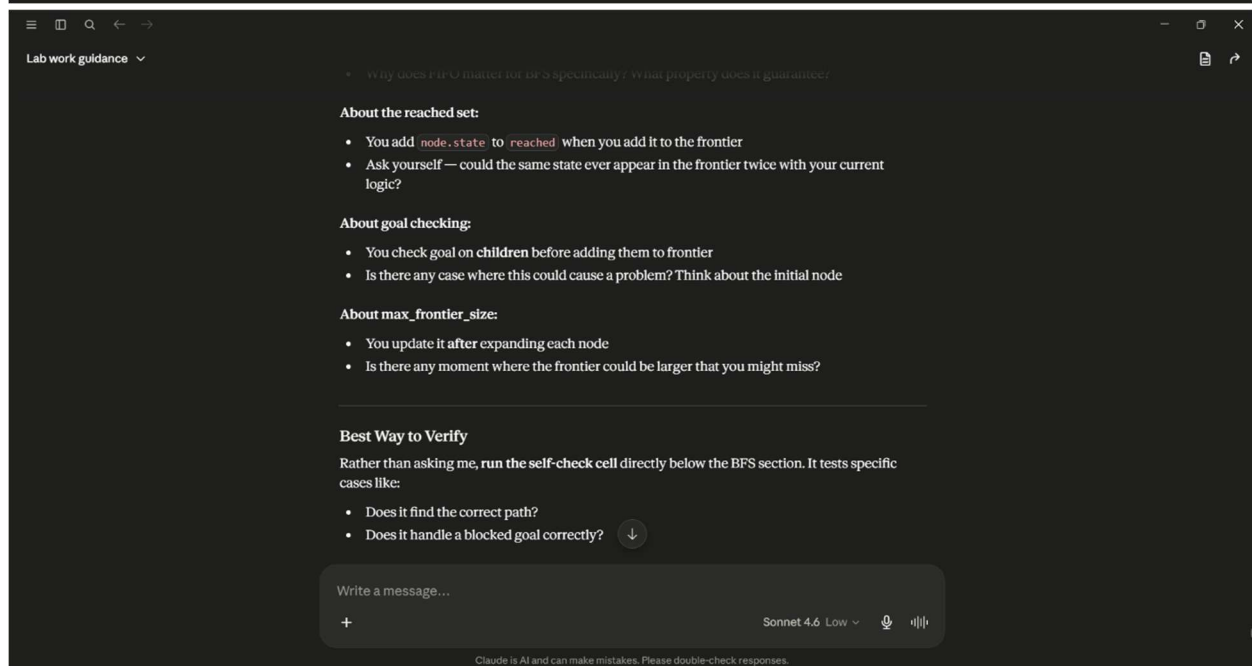
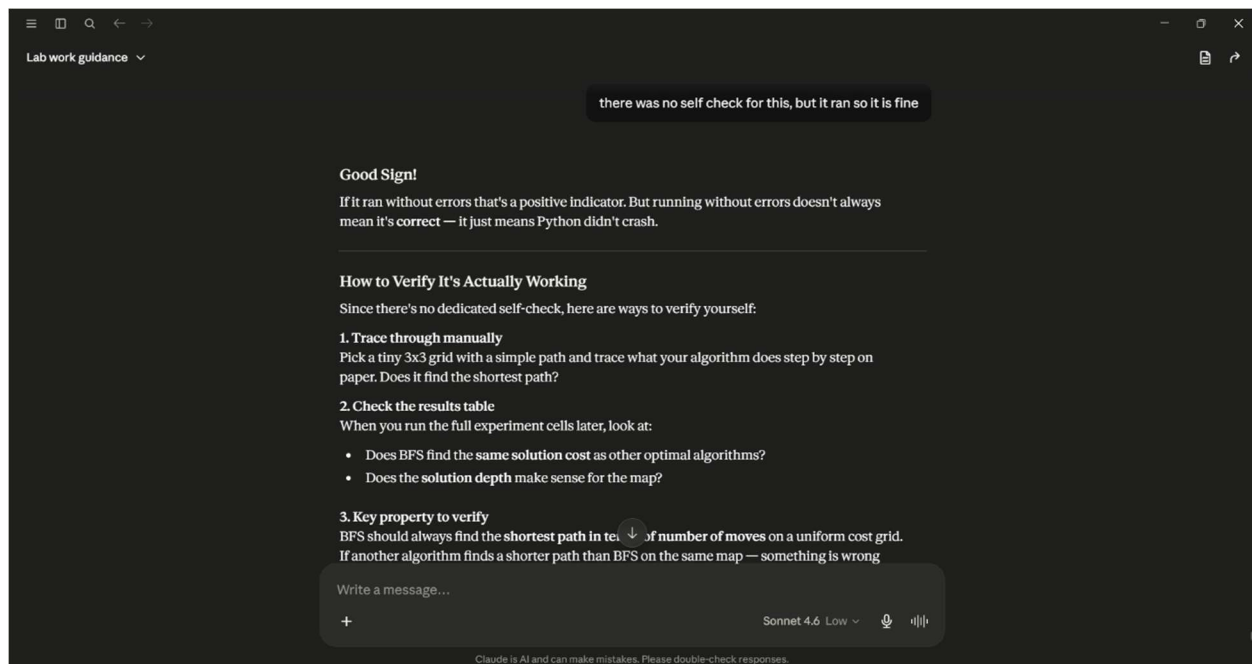
Requirement 1 — Keep a list called Iteration_log

Think of this as a diary of each attempt. Every time IDS runs DLS with a new depth limit, it writes down what happened — what the limit was and what the result was. This helps you track the history of all iterations.

Write a message...

Sonnet 4.6 Low

Claude is AI and can make mistakes. Please double-check responses.



≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

```
reached.add(child.state)
frontier.append(child)
max_frontier_size = max(max_frontier_size,
len(frontier))

return SearchResult(algorithm=algorithm, status="failure",
solution=None, nodes_expanded=nodes_expanded,
max_frontier_size=max_frontier_size)
```

this is like the concept in the part A, check for me if i am right

Show less

Looking at Your BFS Implementation

I won't confirm or deny if it's correct — but I'll help you think through it conceptually so you can verify it yourself!

Questions to Ask Yourself

About the queue behaviour:

- You're using `popLeft()` — does that give you FIFO (first in, first out)? Think about what end you add to vs what end you remove from
- Why does FIFO matter for BFS specifically: ... what property does it guarantee?

Write a message...

+ Sonnet 4.6 Low 🔊 🗣️

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

2. The `reversed()` call

You're reversing children before pushing them onto the stack. Think about why that might or might not be necessary. What order do children come out of `expand`? What order do you want them explored?

3. Structure looks solid

The overall structure — initial check, frontier as list, pop from end, expand, update max size, return failure — follows the pseudocode correctly.

How to Test It

Rather than guessing, run the self-check cell underneath the DFS section. It will tell you immediately if something is wrong. If you get an `AssertionError`, paste it here and I'll explain what that specific error means conceptually.

Does the logic make sense to you when you trace through it manually with a small example?

↓ Explain each section of part B and what it means

Write a message...

+ Sonnet 4.6 Low 🔊 🗣️

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

```
max_frontier_size=max_frontier_size)
    if child.state not in reached:
        reached.add(child.state)
        frontier.append(child)
        max_frontier_size = max(max_frontier_size,
                                len(frontier))

    return SearchResult(algorithm=algorithm, status="failure",
                        solution=None, nodes_expanded=nodes_expanded,
                        max_frontier_size=max_frontier_size)
```

ok so check this out and see if there are errors

Show less

10:14 PM ↻ ✎ 📄

I can see your implementation! Remember our agreement — I won't tell you exactly what to fix, but I'll point out concepts to think about.

Things to Think About

1. The `reached` set and goal checking order

Look at your loop carefully. You check `is_goal` on children **before** adding them to `reached`. But what about a child that isn't the goal — you ↓ it to `reached` when you add it to the frontier.

Write a message...

+

Sonnet 4.6 Low ▾ 🗣️ 🔊

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

3. Node Expansion

When you "expand" a node you generate its children — all possible next states from the current position. Research:

- What information does a child node need? (state, parent, action, cost)
- Why do we track the parent node?

4. Frontier Management

The frontier is the collection of nodes waiting to be explored. Research:

- What happens if the frontier becomes empty before finding the goal?
- What does `max_frontier_size` tell you about memory usage?

5. Recursion vs Iteration

DFS can be written both ways. This lab uses the iterative approach with a stack. Research:

- Why might iterative be preferred over recursive for large grids?

↓

Start with stacks — everything else builds on that. What concept feels most unclear right now?

Write a message...

+

Sonnet 4.6 Low ▾ 🗣️ 🔊

Claude is AI and can make mistakes. Please double-check responses.

Lab work guidance

what concepts should I learn to understand this

10:13 PM

Core Concepts to Understand DFS

1. Stacks (Last In, First Out)

Think of a stack of plates — you always take from the top. In Python a list acts as a stack. Research:

- How `append()` adds to the end
- How `pop()` removes from the end
- Why this order matters for DFS behaviour

2. Graph Search vs Tree Search

DFS can get stuck in loops if you revisit states. The `reached` set prevents this. Research:

- What is the difference between graph search and tree search?
- Why does revisiting states waste time or cause infinite loops?

Write a message...

Sonnet 4.6 Low

Claude is AI and can make mistakes. Please double-check responses.

Lab work guidance

Breaking Down Each Step

Step 1 — Create the initial node

Think about what information a node needs to store. Where does the drone start? Look at how other search methods in the notebook create their first node.

Step 2 — Check if already at goal

Before searching at all, what if the drone is already at the destination? This is an edge case check. What should you return if true?

Step 3 — Stack frontier

A stack in Python is just a regular list. The key thing about a stack is **last in, first out** — meaning you always take from the end. How is that different from BFS's queue?

Step 4 — Reached set

Why a set and not a list? Think about what you need to check quickly — "have I visited this state before?" A set makes that check very fast.

Step 5 — The main loop

- `pop` from end — which Python list method does this?
- expand the node — there's already an `expand` method in the class, how do you call it?
- unreached children — before adding a child, what condition must be true?
- `max_frontier_size` — you're tracking the largest the frontier ever gets, how would you update that?

Write a message...

Sonnet 4.6 Low

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

Steps:

1. Create the initial node.
2. If the initial state is the goal, return success.
3. Use a Python list as the stack frontier.
4. Use a reached set.
5. While frontier is not empty:
a. pop from the end of the list.
b. increment nodes_expanded.
c. expand the node.
d. add unreached children to the stack.
e. update max_frontier_size.

for the todo 7 for PartA, how should i go about it

Show less

Remember our agreement — I won't give you implementations! But I can help you understand what each step is asking conceptually so you can write it yourself.

Breaking Down Each Step

Step 1 — Create the initial node

Think about what information a node needs to. ⬇ ⬆ Where does the drone start? Look at how other search methods in the notebook create their first node.

Write a message...

+

Sonnet 4.6 Low ▾ 🔊 🗣

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

What Makes a Map Interesting

Think about placing obstacles that create:

1. Multiple possible routes — so algorithms choose differently

2. Dead ends — corridors that look promising but go nowhere, this really exposes DFS weaknesses

3. A non-obvious shortest path — where the direct route is blocked and you have to go around

What to Think About

Ask yourself:

• If I were a drone, which way would look most direct?

• What if I blocked that obvious route — where would I have to go instead?

• Which algorithm do I expect to get trapped and which do I expect to find the optimal path?

A good map tells a story about why one algorithm beats another on a specific metric — nodes expanded, path length, or memory used.

Want to think through what your map should look like before you build it?

⬇

Write a message...

+

Sonnet 4.6 Low ▾ 🔊 🗣

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

I tried making my custom map just going straight down column 0 from (0,0) to (14,0) then right to (14,14). It found a path but it looks boring. Would it be better to make it zigzag so the algorithms have more decisions to make?

Great Observation!

Yes, a straight path is boring for a very important reason — if there's only one possible route, all algorithms will find the same path. That makes it impossible to see any difference between BFS, DFS, IDS etc.

Why Zigzag is Better

A good custom map should force the algorithms to make real decisions at branching points. Think of it like this:

- Straight path → every algorithm succeeds the same way, no interesting comparison
- Zigzag with obstacles → some algorithms get trapped in dead ends, some find shortcuts, some take longer routes

What Makes a Map Interesting

Think about placing obstacles that create:

Write a message...

+ Sonnet 4.6 Low 🔊 🗣️

Claude is AI and can make mistakes. Please double-check responses.

≡ □ 🔍 ⬅ ➡

Lab work guidance ▾

Data and visualisation:

- `import numpy as np` — for working with grid arrays efficiently
- `import pandas as pd` — for displaying results in a clean table
- `import matplotlib.pyplot as plt` — for drawing the grid and solution paths visually
- `import matplotlib.patches as patches` — for drawing the coloured rectangles on the grid

Why does all this matter?

Without these, you'd have to build everything from scratch. They let you focus on the actual search logic rather than reinventing basic data structures.

What would you like to understand next?

I tried making my custom map just going straight down column 0 from (0,0) to (14,0) then right to (14,14). It found a path but it looks boring. Would it be better to make it zigzag so the algorithms have more decisions to make?

Great Observation!

Yes, a straight path is boring for a very important reason — if there's only one possible route.

Write a message...

+ Sonnet 4.6 Low 🔊 🗣️

Claude is AI and can make mistakes. Please double-check responses.

[illegible]